Mini-Project 2: Motion Analysis and Plotting

Goals

In this mini-project you will build on your logging script from mini-project 1 to develop an application which can perform basic analysis on data collected from an accelerometer. Concepts from lecture which will be applied in this mini-project include:

- Array Functions
- Array Indexing
- Plotting and annotation

You will also explore basic file input/output and techniques for sharing data with your collaborators. The lab will require you to ultimately produce a number of plots. An example of the desired plot format is given in Figure 1 at the end of this lab manual.

Steps

Step 1: Analysis of Your Own Data

Step 1 will guide you through a basic data analysis pipeline using data collected from your own smartphone.

Background: File I/O

As engineers, we routinely need to share data with each other. For example, we may collect measurements on one device, but then analyze them using a more powerful computer. File input/output (abbreviated I/O) refers to the process of writing data from a program to a permanent file on disk (file output) and reading that data back in a new program (file input).

A simple example is saving a word document to a .docx file on your computer hard drive so that it persists after you close word and then loading that document again later.

File I/O in Matlab: Matlab provides a simple way to read/write data using the **save** (https://www.mathworks.com/help/matlab/ref/save.html) and **load** (https://www.mathworks.com/help/matlab/ref/load.html) functions.

Step 1.1: Data Collection

Create a copy of the sensor logging script you made for mini-project 1 and modify it to collect 15 seconds of data from your smartphone's accelerometer. Name your script something informative about its purpose like "dataCollection.m."

Next, use the **save** function introduced above to create a .mat file called sensorData.mat which contains a single array called "sensorData" where:

- Columns 1-3 contain, respectively, the measured X, Y, Z acceleration values.
- The fourth column contains the *timestamp* of the sample.

Important: your scripts will be submitted to the autograder and *must* use the file/variables stipulated in the lab manual. If a variable/file name is not stipulated, you may use whatever name you want.

Step 1.2: Data Analysis

Create a **new** script called dataAnalysis.m which performs the following functions.

1) Uses the **load** function to read the file you wrote in the previous step.

2) Using the data you collected create plots with time on the X axis and the following quantities on the Y axis.:

- The magnitude of acceleration in the X, Y, and Z directions.
- The *resultant acceleration* - the resultant acceleration can be calculated by the formula:

$$A_{resultant} = \sqrt{A_x^{\ 2} + A_y^{\ 2} + A_z^{\ 2}}$$

Where A_x, A_y, A_z is the acceleration in the X, Y, and Z directions respectively?

Your plot should be **one** panel with four subplots. **Hint**: to create a subplot, use the **subplot** function (https://www.mathworks.com/help/matlab/ref/subplot.html). You can modify/extend the plotting code from mini-project 1 for this purpose.

Reminder: to call an external script, it must be in the same directory.

Step 1.2: Peak Detection

Peak detection is a fundamental technique in signal processing and refers to the problem of finding the point(s) in time at which a measured signal attains its maximum value. To give a concrete example: consider a temperature sensor measuring water being heated up for tea. The peak value of the signal is simply the hottest temperature measured. Peak detection refers to the problem of finding the time at which the water was hottest.

In “dataAnalysis.m,” write code which will calculate the time-elapsed to the peak of the acceleration magnitude in the X, Y, Z directions as well as the resultant acceleration. That is, if the maximum acceleration magnitude in the X direction occurs after 13.25 seconds of data collection, you should produce a variable called (for example) *peakTimeX* which has the value 13.25.

Step 1.3: Plot Annotation

Extend your plotting code as follows:

- For each plot, add a vertical red line at the time corresponding to peak acceleration-magnitude.
- For each red line annotate it clearly with text indicating the peak magnitude. **Hint**: to annotate a plot in Matlab with text, use the **text** (https://www.mathworks.com/help/matlab/ref/text.html) function. You can use the **string** (https://www.mathworks.com/help/matlab/ref/string.html?searchHighlight=string&s_tid=srchtitle_support_results_1_string#d126e1747207) function to convert the numeric acceleration value into a string for annotation.
    - You should offset your annotation slightly from the vertical line so that it is more clearly visible.
    - Recommended but not required: Consider adding a small amount of vertical space above the plot so that the annotation does not interfere with the plot.

An example is shown in Figure 1 below. The final output of your script dataAnalysis.m should be:

- 1 PNG file which contains the saved figure with annotation described above. You must name your file “accelerationPeaks.png.” **Hint**: use the **saveas** (https://www.mathworks.com/help/matlab/ref/saveas.html) or **exportgraphics** (https://www.mathworks.com/help/matlab/ref/exportgraphics.html) functions to save a matlab figure as a PNG file. The latter provides more fine-grained control than the former.
- A 1x4 array called “peakTimes” which contains the elapsed time until the peak acceleration magnitude in the X, Y, Z directions (items 1-3) and the resultant acceleration (item 4)

In your lab report, include the final output PNG file with annotation.

Step 1.4 (Optional)

Producing visually appealing and clear graphics is an important skill. Humans are (in general) highly visual creatures and the design of figures can make-or-break a presentation or publication.

Try uploading your PNG file to an AI tool like ChatGPT and ask for suggestions on how the quality of your plot could be improved. Try integrating some of these suggestions into your plot. Do you like the result?

Step 2: Data Exchange and Peer Validation

To validate your code, send the data you collected in part 1.1 to a partner. To send data, you can use email, GoogleDrive, AirDrop, telegraphy, or any other transmission technique you prefer (binary coded messages tied to a pigeon). Re-run your code on your partner's data and verify you calculated the same peak times as them in part (1).

In your lab report include copies of the plots you reproduced of your partner's data. **Note**: you must produce your *own* versions of your partner's plots. You must include the name of your partner in your lab report.

Example Plot

Figure 1 below shows an example of what your plot should look like. Your plot should generally follow the format of this example but you're not required to match it exactly. You're free to make modifications and extensions if you wish, but plots which are excessively difficult to interpret or visually unappealing may lose points.

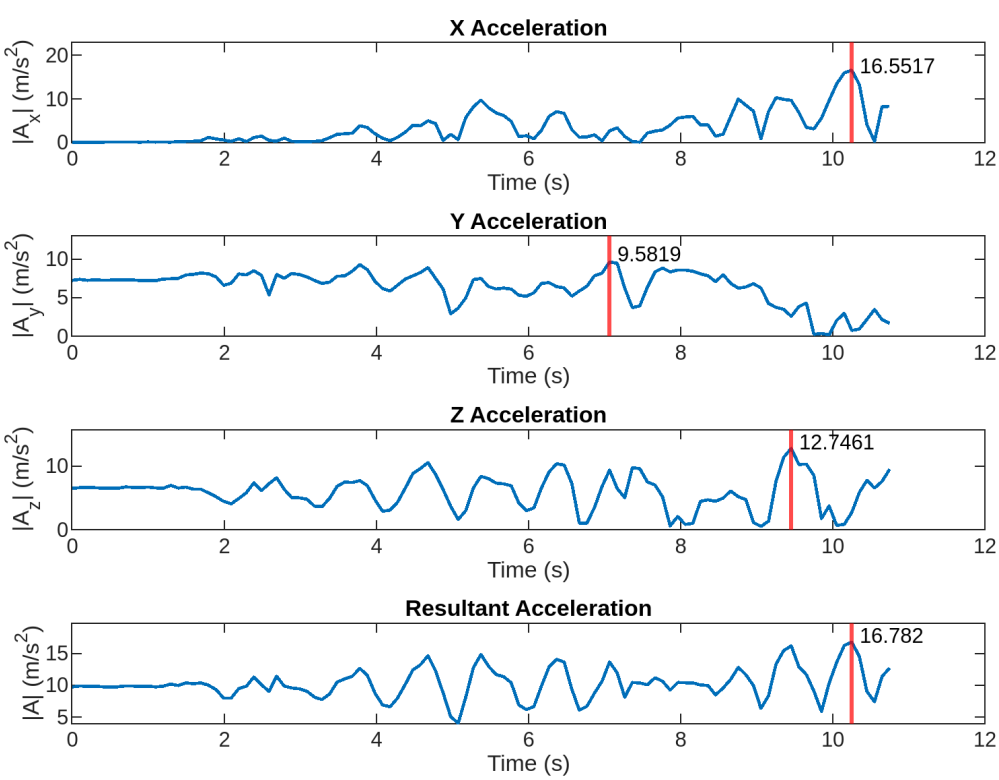

Figure 1: Example plot. Note the use of slight vertical padding to ensure the annotation does not spill into the line being plotted.

What to Submit

Submit your dataAnalysis.m script to the autograder on Canvas. The autograder will run your script on a reference data file and verify that you correctly calculate peak times and produce plots.

Submit a lab report containing:

- A short summary of the project
- The annotated plot of your *own* data
- The annotated plot of your partner's data along with their name.

Note: it is a violation of the academic integrity policy to submit someone else's plots as your own. We will run your code. If you submit plots, but your code does not run, we will be suspicious.

Rubric

- **Code**: `dataAnalysis.m` runs without errors, correctly displays peakTimes, and produces annotated plots.
- **Plots**:
    - Plots are clear, identify peaks with a red vertical line, and include clear annotations at the peaks.
    - Both annotated plots, showing your own and your partner's collected data, are generated from your own code.
    - Annotated plot of partner's data must include partner's name in lab report.
- **Reflection**:
    - Reflection meaningfully discusses AI results and how results were verified, and/or commentary on if it greatly differs from functions being used in class.